TP2 REPORT

Requirements Engineering & Use Case Modeling

Corrected and completed version

Prepared from the provided TP2 Requirements Specification and report.

Exercises covered

• Exercise 1 — Productivity and Lines of Code

• Exercise 2 — Ticket Distributor System

• Exercise 3 — Recruitment Web System

• Exercise 4 — Hotel Management Software

The diagrams supplied by the student are included in the appropriate exercises.

Exercise 1 — Productivity and Lines of Code

1. Time required when working alone

Firefox: 10,000,000 lines ÷ 100 lines/day = **100,000 days** ≈ **274 years**.

Google: 2,000,000,000 lines ÷ 100 lines/day = **20,000,000 days** ≈ **54,795 years**.

These results assume one person continuously produces 100 correct lines of code every day, without accounting for maintenance, testing, requirements, or other software-engineering activities.

2. How can companies succeed despite such enormous amounts of code?

• Large teams work in parallel on different components.

• Software is divided into modules and components that can be developed and maintained independently.

• Developers reuse existing libraries, frameworks, and code instead of implementing everything from scratch.

• Teams use standard processes, version control, testing, code review, and collaboration practices.

3. Is lines of code a good measure of productivity?

No. A developer who writes 500 lines is not necessarily more productive than one who writes 50 lines. The shorter solution may be better if it is simpler, correct, maintainable, efficient, and has fewer bugs.

Better ways to evaluate productivity:

• Quality and number of defects/bugs.

• Completion of useful features and requirements.

• Maintainability, readability, and simplicity of the code.

• Test coverage and reliability.

• Ability to collaborate, review code, and deliver work on time.

4. What remains as the unique contribution of human developers when AI generates most code?

Human developers remain responsible for understanding the real problem and users' needs, making architectural and ethical decisions, validating requirements, reviewing AI-generated code, identifying risks, and applying critical thinking. AI-generated code can contain mistakes, so human verification remains essential.

Exercise 2 — Ticket Distributor System

Actors:

• Traveler — purchases different types of tickets.

• Central Computer System — maintains the reference database and supports tariff updates.

Use cases:

• BuyOneWayTicket

• BuyWeeklyCard

• BuyMonthlyCard

• UpdateTariff

Use Case Diagram — Ticket Distributor

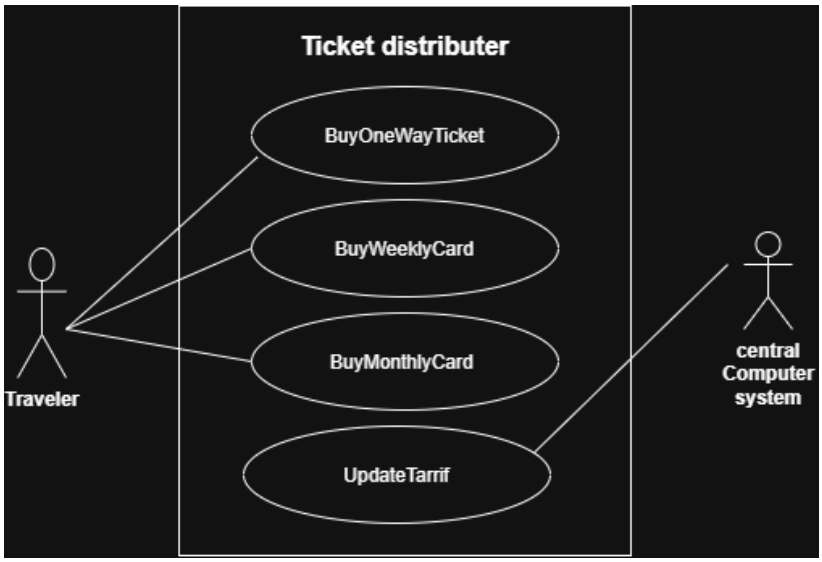


Figure 1. Student-provided use case diagram for the ticket distributor system.

Exercise 3 — Recruitment Web System

The following five functional requirements cover the main workflow described in the specification. The effort estimates are proposed estimates because the assignment asks for an estimate but does not provide fixed values.

ID	Functional requirement	Priority	Estimated effort
FR1	The system shall allow a job applicant to create and validate a CV using the CV Builder module.	Must have	1 week
FR2	The system shall allow the career expert to review a validated CV and send recommendations when gaps are identified.	Must have	2–3 weeks
FR3	The system shall allow the career expert to add a suitable CV to a shortlist for a specific job position and submit the shortlist to the client manager.	Must have	1–2 weeks
FR4	The system shall allow the client manager to create automated tests, invite shortlisted candidates, evaluate results, and send the appropriate notifications.	Must have	2–3 weeks
FR5	The system shall send weekly job announcements to users for positions matching their profiles, preferences and ongoing job postings.	Should have	2 weeks

Use Case Diagram — Recruitment Web System

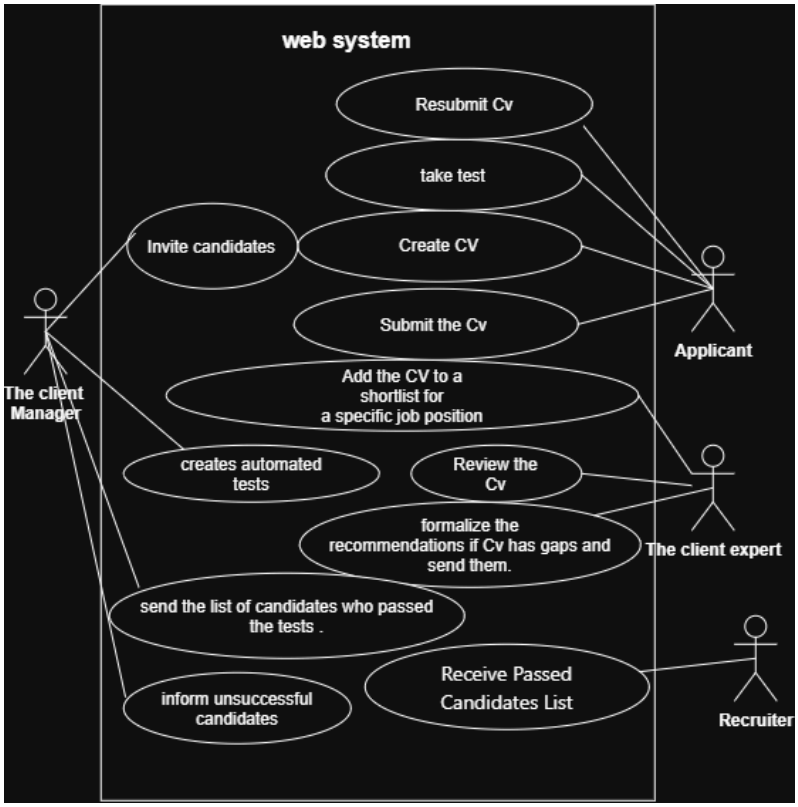


Figure 2. Student-provided recruitment web system use case diagram.

Exercise 4 — Hotel Management Software

1. Actors of the system

• Hotel client — makes reservations and may add consumptions during the stay.

• Receptionist — finds rooms, verifies reservations, performs check-in/check-out, and receives payment.

• Cashier — adds consumptions to the invoice through the Billing module.

• Cleaning staff — cleans rooms and marks rooms as “clean waiting.”

• Floor manager — verifies cleaning and marks rooms available or unavailable.

• Client's bank server — validates online CIB-card payments; communication must use SSL.

2. Actions that occur outside the system

• A reservation made by phone is outside the software system.

• The client's bank server validates the online payment outside the hotel application; the communication with it uses SSL.

• The physical cleaning of rooms is an activity performed outside the software, although the cleaning status is recorded in the system.

• Maintenance intervention to repair a damaged room is performed outside the software; the system records the room as unavailable until the intervention is completed.

3. Functional boundaries / modules

• Module 1 — Reservation: make a reservation.

• Module 2 — Payment: validate online payment.

• Module 3 — Reception: verify reservation, find an available room, perform check-in, receive payment, and close the stay.

• Module 4 — Billing: add consumptions to the invoice.

• Module 5 — Cleaning: clean rooms, mark rooms as “clean waiting,” and verify/record cleaning status.

• Module 6 — Maintenance: mark a room unavailable and available again after maintenance.

4. Use Case Diagram with Functional Boundaries

The provided hotel-management diagram is placed below because it represents the six functional boundaries identified above.

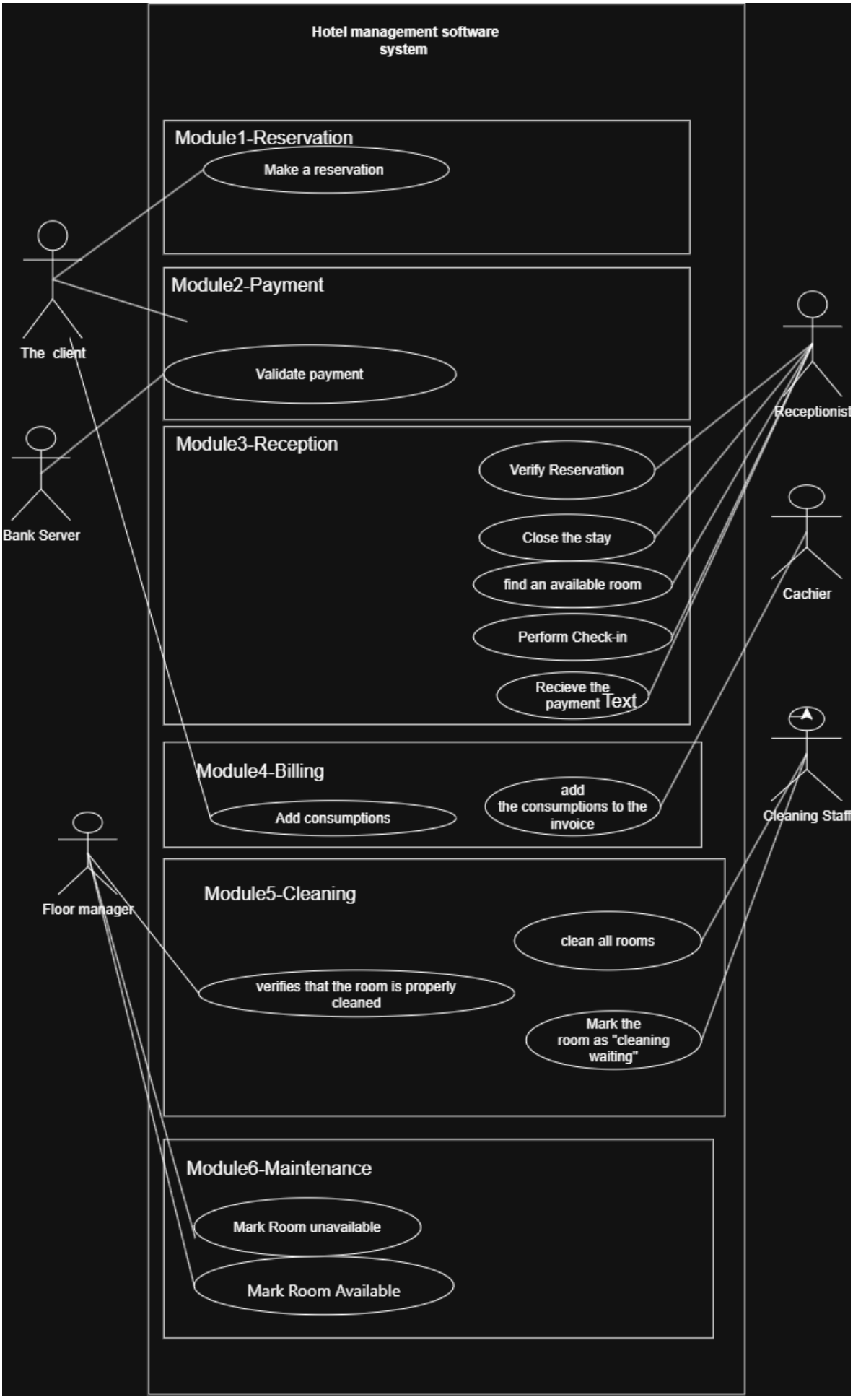


Figure 3. Student-provided hotel management software use case diagram.

Summary of corrections

• Grammar, spelling, punctuation, and mathematical notation were corrected.

• Exercise 2 was completed with the actors, use cases, and the supplied Ticket Distributor diagram.

• Exercise 3 was completed with five functional requirements, priorities, effort estimates, and the supplied Recruitment Web System diagram.

• Exercise 4 questions 1–3 were answered and the supplied hotel-management diagram was inserted under question 4.

• The result is organized as one report suitable for submission and versioning in a GitHub repository.